



RAG Beyond the Demo

How we turned a promising retrieval-augmented generation prototype into a insurance claims adjudication system – with the evaluation, guardrails, and operational patterns that actually made it work.

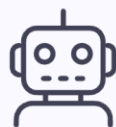


AGENDA

Agenda



RAG
Foundations



Inside
Domain
Copilot



Real-World
RAG



Security &
Access



Evaluation &
Lessons

Case Study

Domain Copilot: RAG in Insurance Claims Adjudication

Today, we'll explore how Retrieval-Augmented Generation (RAG) operates within a real-world insurance claims adjudication system. We'll meticulously follow the journey from:

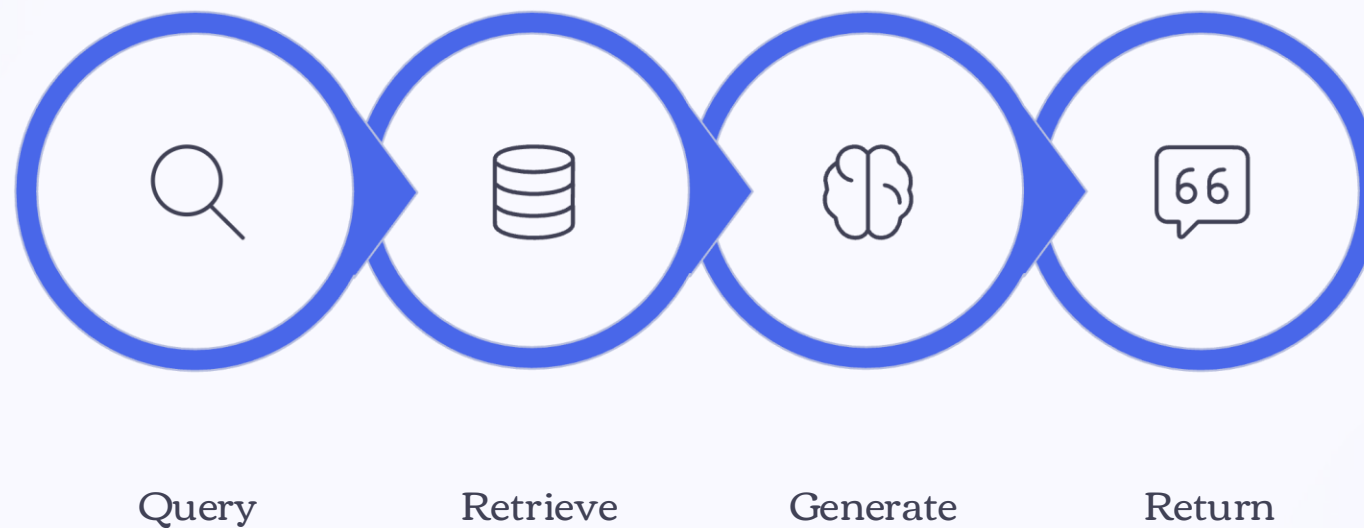
Document Ingestion & Retrieval · Grounded Answers · Failures & Debugging · Security & Guardrails

Instead of just theoretical discussions, we will dive into the **real bugs** we encountered and the **real fixes** we implemented to ensure system reliability and accuracy.

What RAG Actually Is

Retrieval-Augmented Generation means: first retrieve the most relevant text, then generate an answer constrained by that text. The LLM does not "know" your documents in advance – you surface the right context at query time. There are no weight changes and no fine-tuning required.

Key insight: RAG is really about **separation of concerns**. The retriever is responsible for finding useful context; the LLM is responsible for reasoning over it. That division is powerful, but it also means production RAG can fail in many places: retrieval quality, chunking, ranking, context limits, and generation all matter.



The retriever acts like a librarian, finding the right pages. The LLM acts like a subject-matter expert, reading those pages and answering the question in context.

Why the Basic RAG Isn't Enough

A demo retrieves *something* relevant once, on a clean query. Production is a different discipline entirely.

Document drift

Wrong policy versions, superseded clauses, conflicting endorsements

Adversarial queries

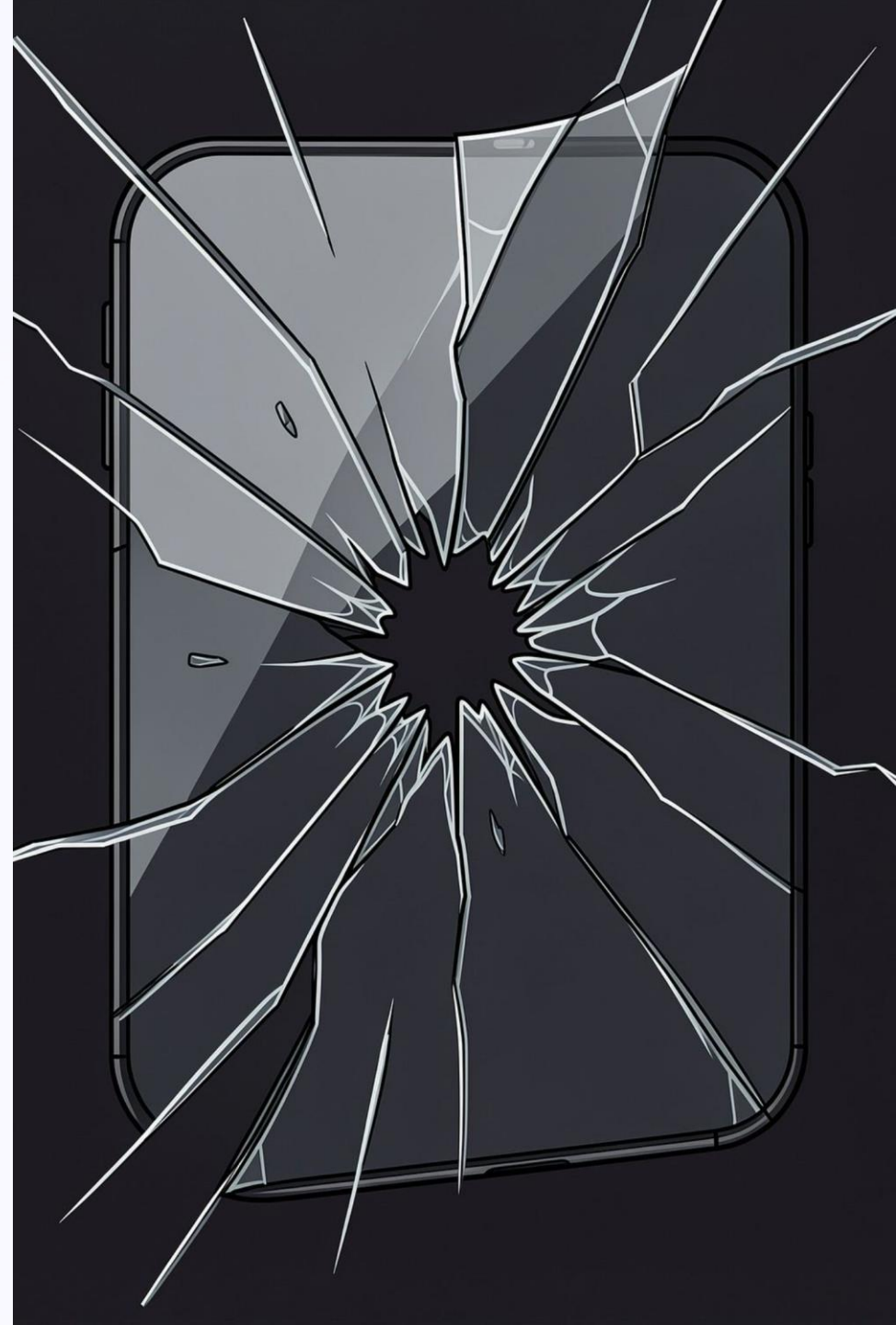
Users who phrase questions to bypass grounding constraints

Partial evidence

When the corpus genuinely lacks an answer, "I don't know" is the correct response

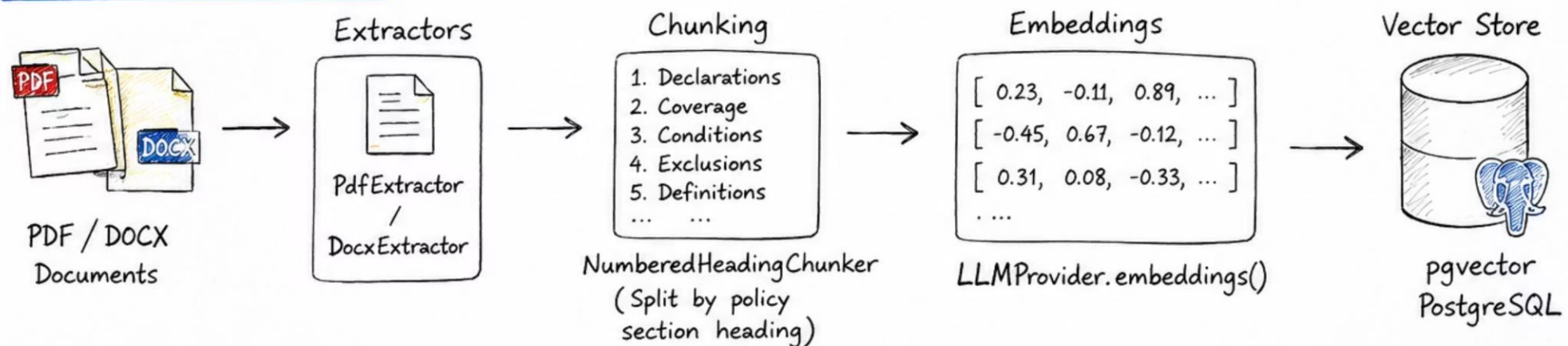
Concurrent load

Multiple users, streaming outputs, shared vector stores under real traffic



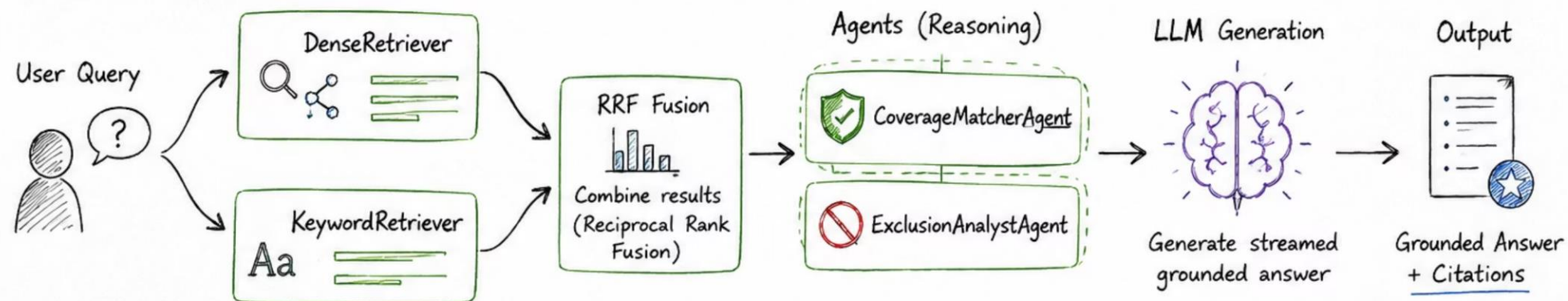
This Project's Real RAG Pipeline

PHASE 1 – INGESTION



Documents are transformed into searchable, policy-aware vector chunks.

PHASE 2 – QUERY & GENERATION



Retrieve → Reason → Generate → Cite

RAG vs. LLM-Only Generation

Dimension	LLM-Only	RAG
Grounding	None – trained knowledge only	Retrieved document text
Confidence	High, even when wrong	Constrained by evidence quality
Policy limits	May confidently state wrong figures	Cited from actual clauses
Payout math	LLM calculates (risky)	Deterministic Python only

The Key Architectural Decision

Use the LLM for reasoning over evidence — not for deterministic business logic.

`calculate_payout()` performs the actual payout calculation in Python, independently of the LLM.

RAG grounds the decision. Deterministic code controls the numbers.

Chunking: Why It Matters Here

```
heading_match = HEADING_RE.match(line)

if heading_match: current_section_num =
heading_match.group(1) current_section_title = (
f"{current_section_num}.
{heading_match.group(2).strip()}" ) clause_counter = 0
```

Numbered-heading chunking

Chunks are split by **numbered section heading** – "1. Declarations", "5. Exclusions" – not by fixed token windows. Citations must map to a real policy clause, not an arbitrary text span.

Real bug discovered

A PDF line-wrap placed "0026. Policy Period: from..." at the start of a visual line. The regex matched it as a section heading, creating a spurious chunk boundary. Fixed by tightening the heading pattern to require a word boundary and a substantive title.

Embeddings: What They Actually Represent

A vector encoding **semantic similarity** – not understanding, not fact-checking. Two embedding models are deliberately decoupled from the completion model.

OpenAI text-embedding-3-small

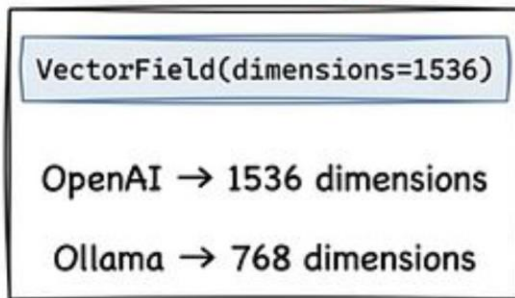
1,536 dimensions – used when OpenAI is the active
`EMBEDDING_PROVIDER`

Ollama nomic-embed-text

768 dimensions – local, self-hosted alternative with a
different vector space

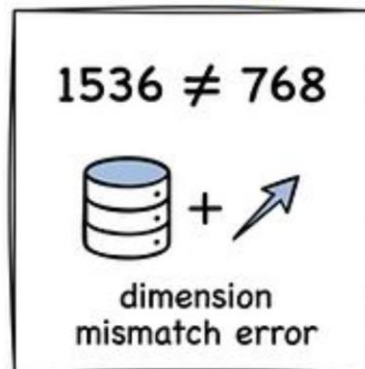
- ❏ `EMBEDDING_PROVIDER` and `LLM_PROVIDER` are independent settings. Swapping one does not require changing the other – but dimension mismatches will break ingestion.

BEFORE ❌



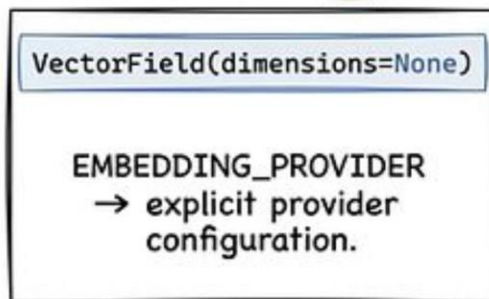
Hard-coded assumption

BUG ⚠️



Dimension mismatch

AFTER ✅



Real Bug: Embedding Dimension Mismatch

When `DocumentChunk.embedding` was defined as `VectorField(dimensions=1536)`, ingesting with Ollama active caused immediate insert failures – the vector length did not match the schema.

1 Root cause

Hard-coded dimension assumed OpenAI; Ollama produces 768-dim vectors

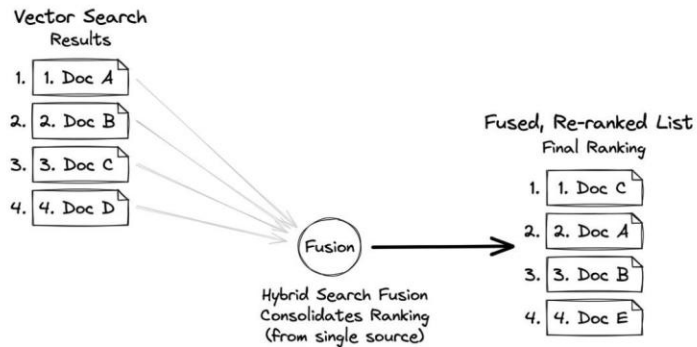
2 The fix

Unconstrained `pgvector` column + `EMBEDDING_PROVIDER` as its own explicit setting, with no fallback chain

3 The principle

Provider assumptions must never be implicit – every model choice is an architectural contract

Hybrid Retrieval



Hybrid Retrieval

Dense Retrieval

Utilises `DenseRetriever` with `pgvector` for semantic search, measuring similarity via cosine distance.

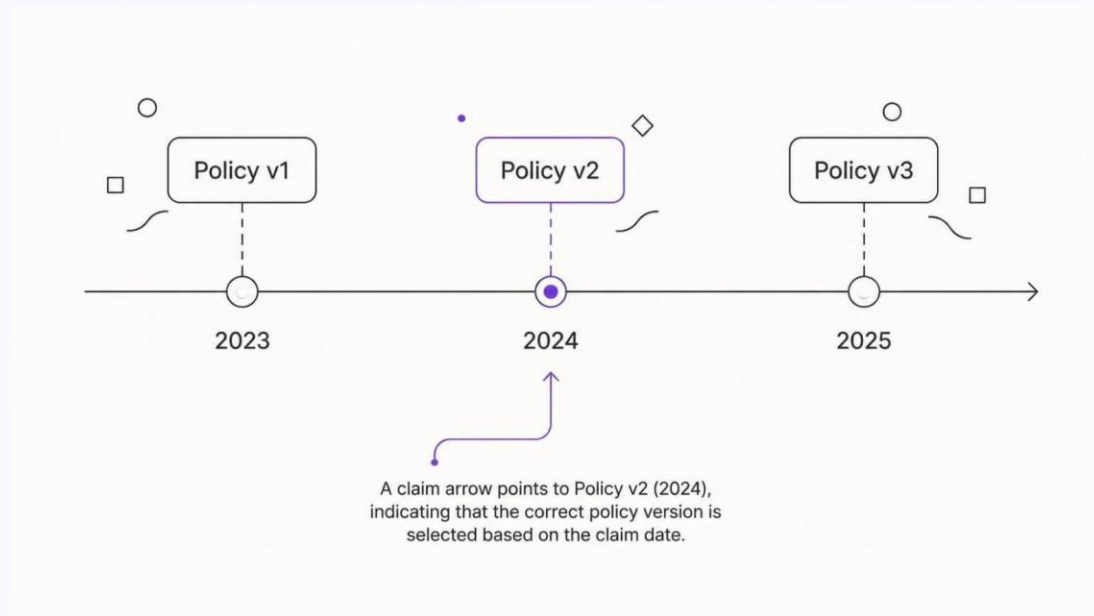
Keyword Retrieval

Employs `KeywordRetriever` leveraging Postgres full-text search (`SearchRank`) for exact term matching.

These two retrieval methods are combined using **Reciprocal Rank Fusion (RRF)**. Unlike a simple weighted average, RRF combines the strengths of both systems by assigning scores based on an item's rank in each list, resulting in a robust, fused list that prioritises highly ranked documents from either source.

Metadata Filtering: The Policy-Version Problem

A critical named domain risk was retrieving the wrong policy version for a claim's specific date of loss, leading to inaccurate information.



To mitigate this:

- A `get_policy_version` tool is invoked to resolve the correct `PolicyVersion` based on the `claim_date` before any retrieval occurs.
- Subsequent retrieval operations are strictly filtered to only search within the documents associated with that specific `policy_version_id`

Boilerplate Ambiguity in Retrieval

Identical or near-identical text across different policy types can lead to incorrect retrieval if not properly contextualized.

Auto Policy - Exclusions

This policy excludes coverage for intentional acts, acts of war, nuclear risks, and damages caused by normal wear and tear.

Cyber Policy - Exclusions

This policy excludes coverage for intentional acts, acts of war, nuclear risks, and damages caused by normal wear and tear.

The Challenge

Multiple synthetic policy types shared near-identical "Exclusions" or "General Conditions" boilerplate.

The Symptom

A query about cyber liability limits incorrectly retrieved the auto policy's limits section.

The Fix

Contextualize each chunk's embedded text by prepending its policy name and section, e.g., "{policy_name} – {section}:"
{content}", while keeping the citation-facing content clean.

Structured Citations & Grounding

Every piece of retrieved information isn't just a block of text; it's a structured object. Each result includes a detailed citation containing its `chunk_id`, source document, specific `policy_version`, relevant `section` and `clause`, page number, and the `excerpt` itself.

A single, consistent `format_citation()` function ensures uniformity across the system, presenting these details clearly to the user.

Crucially, **grounding** means the LLM's response is *constrained* by this retrieved text, not merely provided with it. The model must derive its answer directly from the supplied excerpts, preventing hallucination.

Sample Citation JSON

```
{
  "chunk_id": "pol-xyz-v2.1-sec3.2.1-p42",
  "document": "Commercial_Liability_Policy_v2.1.pdf",
  "policy_version": "v2.1",
  "section": "3.2.1",
  "clause": "Exclusions: Intentional Acts",
  "page": "42",
  "excerpt": "This policy expressly excludes coverage for any loss or damage arising directly or indirectly from intentional acts..."
}
```


Refusal: The Hardest Real Problem in This Project

A binding principle for our system was to **refuse rather than guess** on low-evidence questions, ensuring accuracy over speculative answers.

First Attempt

We initially tried to gate refusal on the fused Reciprocal Rank Fusion (RRF) score of retrieved documents. This approach failed empirically during testing.

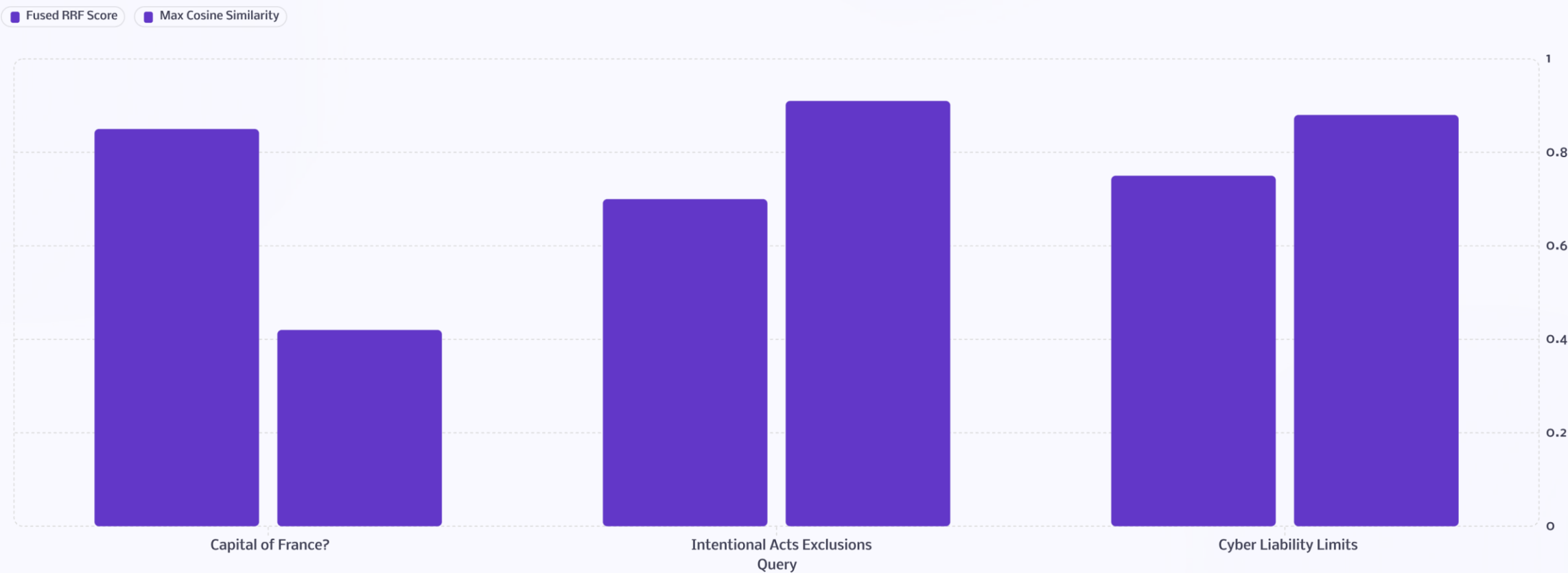
Why it Failed

RRF reflects rank position only; an irrelevant query's nearest (still distant) match can rank #1 in both retrieval lists and score identically to a genuinely strong, relevant match, making it an unreliable signal for confidence.

Real Data: Why RRF-Gated Refusal Failed

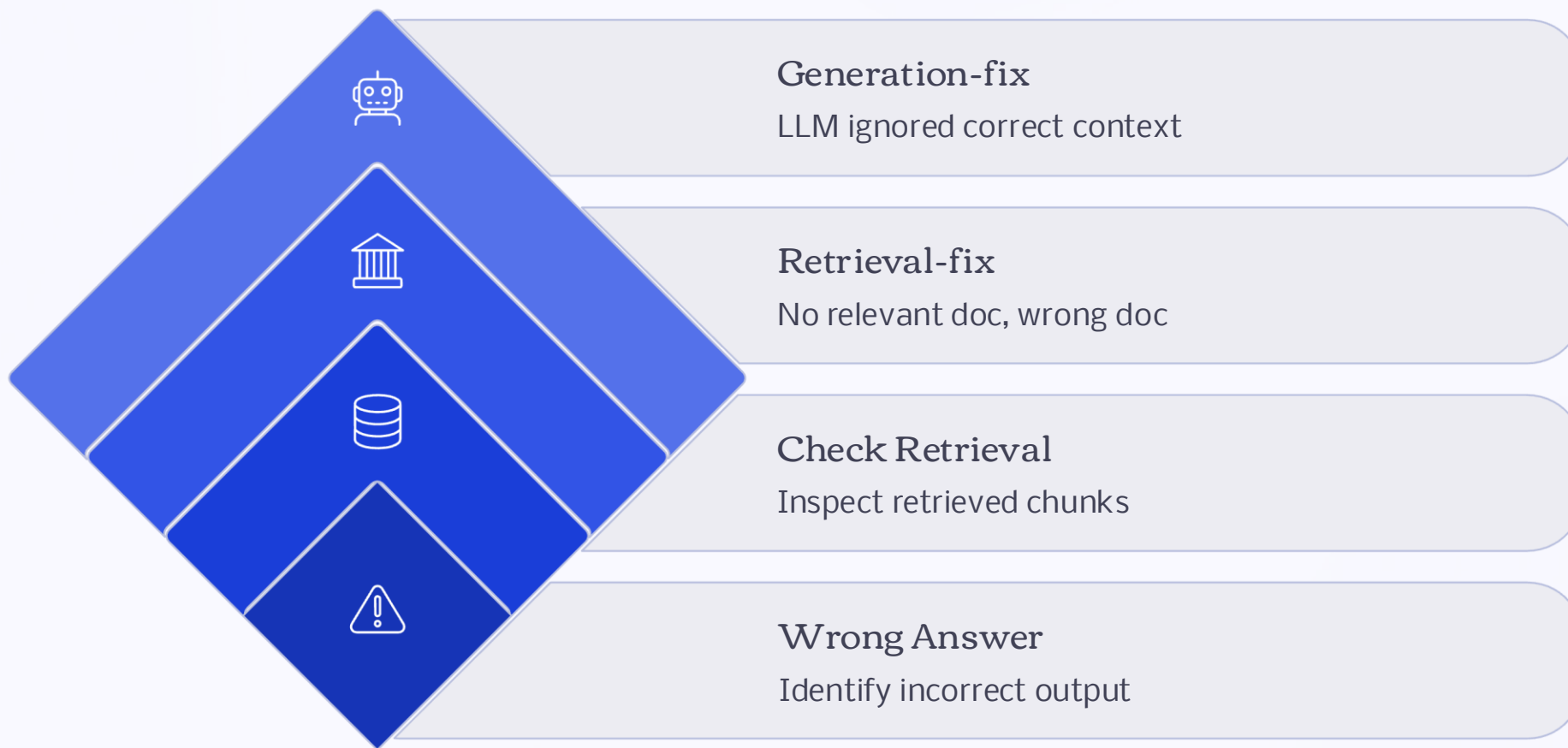
Our initial refusal mechanism, gated on the fused Reciprocal Rank Fusion (RRF) score, proved unreliable in distinguishing genuinely relevant answers from irrelevant but highly-ranked noise.

For example, the query "What is the capital of France?" often received a higher fused RRF score than several questions directly answerable by the policy documents.



Failure Modes: A Taxonomy From Real Incidents

Use this decision tree to diagnose a wrong answer quickly: first verify retrieval, then decide whether the fix belongs in retrieval or generation.



Security: Retrieved Content Is Data, Never Instructions

A fundamental security principle in our RAG system is strict privilege separation. Retrieved content is always treated as inert data, never as executable instructions or system commands.



Privilege Separation

Retrieved chunk text is never interpreted as system instructions, ensuring that malicious content cannot hijack the LLM's behaviour or underlying system.



Adversarial Injection Cases

Evaluation includes a dedicated adversarial prompt injection test case, specifically designed to bypass guards and manipulate the LLM.



Indirect Injection

An indirect injection case, where malicious instructions are embedded within an actual ingested document, was tested and mitigated to prevent subversion through data.



Tool Gateway Enforcement

The `ToolGateway` implements strict per-agent tool allow-lists. Any attempt by an agent to call an out-of-scope tool immediately raises a `ToolNotAllowedError`, preventing unauthorised access or actions.

This multi-layered approach ensures the LLM's operational integrity and prevents any retrieved document from escalating its privileges.

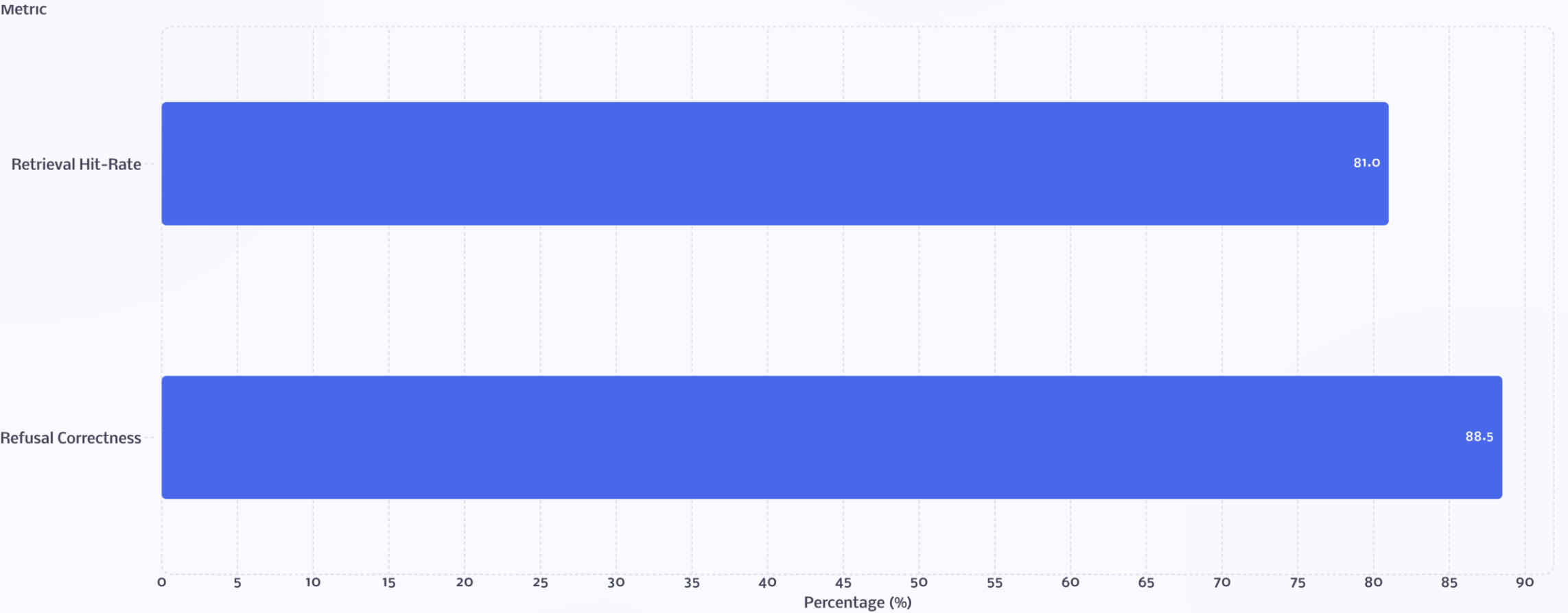
Access Control on Retrieval

Robust access control is paramount to ensure sensitive policy information is only accessed by authorised personnel. Our system implements a fine-grained, role-based access policy at the retrieval layer.

-  **Authentication Layer** User authentication is handled via JWT (JSON Web Token), leveraging Djoser and SimpleJWT, providing a secure and scalable method for user verification.
-  **Defined Roles**
Two distinct roles are established: **ADJUSTER** and **MANAGER**, each with specific retrieval privileges.
-  **Domain-Level Policy**
Access policies are implemented at the domain level (`domain/policies/claim_access_policy.py`), using a framework-free rule set with a thin DRF permission class on top for enforcement.
-  **Role-Based Access**
An **ADJUSTER** is strictly limited to retrieving information pertaining to their **own assigned claims only**. A **MANAGER**, conversely, has the privilege to retrieve information for **any claim** across the system.

Evaluation: Does It Actually Work?

Our system undergoes rigorous evaluation against a comprehensive 26-case **golden set**, comprising 20 normal queries and 6 adversarial scenarios to test resilience. Real-world recorded results demonstrate robust performance, with an **81.0% retrieval hit-rate** and **88.5% refusal correctness**. Crucially, every single miss is meticulously analyzed and documented, ensuring continuous improvement and transparency rather than being hidden.



Recap and Assessment

As we conclude the theoretical overview, let's consolidate the key takeaways from this deep dive into production-grade RAG.

Specific Failure Points

RAG in production isn't plagued by one monolithic "hallucination" problem, but rather a chain of real, specific failure points that require precise diagnosis and mitigation.

Empirical Problem Solving

Every successful fix and improvement in this project stemmed from meticulous analysis of real system output, not from theoretical assumptions.

Next Steps: Lab

Now, it's time to apply these insights. You will complete the lab tasks and tackle stretch challenges to build a deeper, practical understanding.

HANDS-ON LAB

INTRODUCTION

Hands-On Lab

Welcome to the practical lab session. Over the next few steps, you will interact directly with the system, performing key operations and confronting a real-world challenge.

All commands and tools you will use today are authentic and reflect our actual environment; no simulated interfaces or invented commands will be used.

Thank You

Questions? Let's connect and continue the conversation!